



Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026



EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE PROJET DE FIN D'ANNÉE

Spécialité: Génie Logiciel

Intitulé

Conception et Développement d'une Application
Web Full Stack **Bloom**

Lieu du stage

EPI

Réalisé par

Ons GAAYA

Encadré par

Dr Wejdene
Slimane

EPI Digital School Sousse — 2025/2026

Département Informatique

Bloom

Application Web Full Stack

Journal Personnel • Boutique en ligne • Panneau d'administration

Rapport de Projet de Fin d'Année

Matière : Développement Full Stack (React + Node.js)

Réalisée par : Ons GAAYA

Encadrée par : Dr. Wejdene SLIMANE

Année : 2025–2026

Dédicace

*“La fleur ne pousse pas en une nuit, mais chaque jour
d’effort
l’amène un peu plus près de sa pleine floraison.”*

À Allah,

Je Lui rends grâce avant toute chose, pour la force, la patience et la sérénité qu’Il m’a accordées tout au long de ce parcours. Qu’Il continue de guider mes pas.

À mes parents,

Ma gratitude pour vous est infinie. Vous êtes ma première école, mon refuge et ma force. Vos sacrifices silencieux, votre amour inconditionnel et votre confiance en moi ont été le carburant de chacune de mes journées de travail. Puisse Dieu vous accorder santé, bonheur et longue vie.

À mon frère,

Merci pour ta complicité et tes encouragements. Tu sais toujours trouver les mots pour me redonner le sourire quand les nuits de code s’éternisent.

À Dr. Wejdene SLIMANE,

À mon encadrante, pour sa rigueur bienveillante, sa disponibilité et la qualité de ses orientations. Votre encadrement a été une boussole précieuse tout au long de ce projet.

À moi-même — Ons GAAYA,

Pour avoir refusé d’abandonner. Pour chaque nuit de code, chaque bug résolu à 2h du matin. Tu as transformé chaque erreur en apprentissage. **Bloom** est le reflet de ta persévérance. Sois fière.

Ons GAAYA

Remerciements

*Avant tout, je remercie **Allah** qui m'a accordé la volonté, la patience et l'intelligence nécessaires pour mener ce projet à terme.*

À Dr. Wejdene SLIMANE, mon encadrante,

Je lui exprime ma plus profonde reconnaissance. Ses conseils avisés, sa disponibilité et la qualité de son suivi ont été déterminants dans l'aboutissement de ce travail. Travailler sous sa direction a été une chance précieuse.

À l'EPI Digital School de Sousse,

Je remercie l'ensemble du corps professoral et administratif pour la qualité de la formation, les ressources mises à disposition et l'environnement d'apprentissage stimulant qu'ils ont su créer.

Aux enseignants du cours Développement Full Stack,

Pour les concepts enseignés tout au long de l'année, qui constituent le socle technique de **Bloom**.

Aux membres du jury,

Je leur exprime ma gratitude pour l'honneur qu'ils me font en acceptant d'évaluer ce travail.

À mes parents et ma famille,

Pour leur soutien moral indéfectible et leurs encouragements constants.

Ons GAAYA

Résumé

Ce projet de fin d'année porte sur la conception et le développement de **Bloom**, une **application web complète** combinant un **journal personnel**, une **boutique en ligne** et un **espace d'administration**, réalisée dans le cadre du cours de Développement Full Stack à l'EPI Digital School de Sousse.

Bloom offre à chaque utilisateur un **espace personnel sécurisé** où il peut écrire ses pensées quotidiennes, **protéger ses contenus sensibles par un code PIN**, parcourir et commander des produits, gérer sa **liste de souhaits**, et recevoir des **notifications en temps quasi-réel**.

L'application est développée avec **React** côté interface et **Node.js / Express** côté serveur, les données étant stockées dans **MongoDB**. Elle intègre un **système de connexion sécurisé**, une **validation des formulaires**, un **panier d'achat**, un **historique de commandes**, un **upload d'images** pour les produits, et un **panneau d'administration complet**.

Mots-clés : React, Node.js, Express, MongoDB, Authentification, Journal Personnel, Boutique en ligne, Administration, Notifications, Upload d'images.

Abstract

This final-year project presents the design and development of **Bloom**, a **full-stack web application** combining a **personal journal**, an **online shop**, and a **complete administration panel**, developed as part of the Full Stack Development course at EPI Digital School Sousse.

Bloom provides users with a **secure personal space** to write daily entries, **protect sensitive content with a PIN code**, browse and order products, manage a **wishlist**, and receive **near-real-time notifications**.

The application is built with **React** on the frontend and **Node.js / Express** on the backend, with data stored in **MongoDB**. It features **secure authentication**, **form validation**, a **shopping cart**, **order history**, **product image upload**, and a **full administration panel**.

Keywords : React, Node.js, Express, MongoDB, Authentication, Personal Journal, Online Shop, Administration, Notifications, Image Upload.

Table des matières

Dédicace	1
Remerciements	3
Résumé	4
Abstract	5
Introduction Générale	1
1 Présentation Générale du Projet	2
Introduction	2
1.1 Présentation de Bloom	2
1.2 Technologies Utilisées	3
Conclusion	4
2 Analyse des Besoins et Spécifications	5
Introduction	5
2.1 Besoins Fonctionnels	5
2.2 Besoins Non Fonctionnels	6
Conclusion	7
3 Conception du Système	8
Introduction	8
3.1 Architecture Générale de l'Application	8
3.2 Organisation de l'Interface — Les Contextes React	10
3.3 Système de Connexion Sécurisée	11
3.4 Protection des Pages	12
3.5 Upload d'Images Produits	12
3.6 Système de Notifications	13
Conclusion	13

4 Réalisation et Mise en Œuvre	14
Introduction	14
4.1 Organisation du Projet	14
4.2 Validation des Formulaires	14
4.3 Expérience Utilisateur — Notifications Visuelles	15
4.4 Gestion de l'Upload d'Image dans l'Administration	15
4.5 Deux Espaces, Deux Systèmes de Connexion	15
4.6 Difficultés Rencontrées et Solutions	16
4.7 Ce que ce Projet m'a Appris	16
Conclusion	17
Conclusion Générale	18

Introduction Générale

Contexte et motivation

Aujourd'hui, de nombreuses personnes cherchent un **espace numérique personnel** pour exprimer leurs émotions, organiser leur quotidien et faire leurs achats en ligne. **Bloom** naît de ce double besoin : une application web qui combine un **journal intime** et une **boutique en ligne**, dans un seul espace sécurisé et agréable à utiliser.

Problématique centrale

Comment concevoir une application web permettant à un utilisateur connecté de **gérer un journal personnel**, de **réaliser des achats en ligne**, de **gérer une liste de souhaits**, et de **recevoir des alertes en temps réel** — tout en séparant clairement l'espace utilisateur et l'espace d'administration ?

Objectifs du projet

Ce projet a pour objectif de développer une application web complète et fonctionnelle, couvrant plusieurs besoins réels. Il s'agit d'abord de permettre à un utilisateur de **créer un compte**, de **se connecter** et d'accéder à un **espace personnel sécurisé**. L'application offre également un **journal personnel** où l'utilisateur peut ajouter des entrées, des photos, des favoris et des entrées secrètes **protégées par un code PIN**. Une **boutique en ligne** est aussi intégrée, avec un panier, un système de commande et une liste de souhaits. Par ailleurs, un **panneau d'administration** permet de gérer les utilisateurs, les produits et les commandes. Enfin, un système de **notifications** informe les utilisateurs et l'administrateur des événements importants en temps quasi-réel.

Structure du rapport

Ce rapport est organisé en **quatre chapitres** : présentation du projet, analyse des besoins, conception du système, et réalisation technique.

CHAPITRE 1

Présentation Générale du Projet

Introduction du chapitre

Ce chapitre présente le contexte, les fonctionnalités et les technologies utilisées dans **Bloom**.

1.1 Présentation de Bloom

Bloom est une **application web full stack** développée avec **React** pour l'interface utilisateur et **Node.js / Express** pour le serveur, connectés à une base de données **MongoDB**. Elle est composée de **trois espaces principaux** : un journal personnel, une boutique en ligne et un panneau d'administration.

L'application a été conçue pour être à la fois **utile** et **agréable** : l'utilisateur dispose d'un espace intime pour ses écrits, d'une boutique pour ses achats, et l'administrateur dispose de tous les outils pour gérer la plateforme.

TABLE 1.1 – Espaces fonctionnels de Bloom

Espace	Fonctionnalités
Journal Personnel	Écrire, modifier et supprimer des entrées ; ajouter des photos ; marquer des favoris ; protéger des entrées secrètes par code PIN ; recevoir des notifications
Boutique	Parcourir les produits, ajouter au panier , passer commande, consulter l' historique , gérer une liste de souhaits
Administration	Gérer les utilisateurs, les produits (avec photos), les commandes et les notifications
Profil	Modifier sa biographie, son avatar et son code PIN

1.2 Technologies Utilisées

Le choix des technologies a été guidé par les enseignements du cours et par la volonté de construire une application **moderne** et **maintenable**.

TABLE 1.2 – Technologies utilisées dans Bloom

Rôle	Technologie	Description
Interface utilisateur	React + Vite	Construction des pages et composants
Navigation	React Router	Gestion des pages et des redirections
Formulaires	react-hook-form	Saisie et validation des données
Requêtes serveur	Axios	Communication avec le backend
Serveur	Node.js + Express	Traitement des requêtes
Base de données	MongoDB	Stockage des données
Connexion sécurisée	JWT + bcrypt	Authentification et mots de passe
Upload d'images	Multer	Téléversement des photos produits
Style visuel	Tailwind CSS	Mise en page et design

Conclusion du chapitre

Ce chapitre a présenté l'application **Bloom**, ses trois espaces fonctionnels et les technologies qui la composent.

CHAPITRE 2

Analyse des Besoins et Spécifications

Introduction du chapitre

Avant de développer l'application, il est important d'identifier clairement ce qu'elle doit faire (**besoins fonctionnels**) et comment elle doit se comporter (**besoins non fonctionnels**).

2.1 Besoins Fonctionnels

Les **besoins fonctionnels** décrivent les actions que l'application doit permettre à ses utilisateurs. Le tableau ci-dessous liste les principales fonctionnalités de **Bloom**, classées par ordre de priorité.

TABLE 2.1 – Besoins fonctionnels de Bloom

ID	Prio.	Fonctionnalité	Espace
EF01	1	Créer un compte utilisateur	Authentification
EF02	2	Se connecter et rester connecté	Authentification
EF03	3	Écrire, modifier et supprimer des entrées de journal	Journal
EF04	4	Ajouter des photos aux entrées	Journal
EF05	5	Marquer des entrées en favoris	Journal
EF06	6	Protéger des entrées par un code PIN	Journal
EF07	7	Recevoir des notifications en temps quasi-réel	Journal + Boutique
EF08	8	Parcourir les produits de la boutique	Boutique
EF09	9	Ajouter au panier et passer commande	Boutique
EF10	10	Consulter l' historique de ses commandes	Boutique
EF11	11	Gérer sa liste de souhaits	Boutique
EF12	12	Modifier son profil (bio, avatar, PIN)	Profil
EF13	13	Gérer utilisateurs, produits et commandes	Administration
EF14	14	Téléverser des photos pour les produits	Administration
EF15	15	Recevoir une alerte admin à chaque ajout à une wish-list	Administration

2.2 Besoins Non Fonctionnels

Les **besoins non fonctionnels** concernent la qualité et le comportement général de l'application : sa **sécurité**, ses **performances**, et sa **facilité d'utilisation**.

TABLE 2.2 – Besoins non fonctionnels de Bloom

ID	Catégorie	Description
ENF01	Sécurité	Les mots de passe ne sont jamais stockés en clair
ENF02	Sécurité	Les pages privées ne sont accessibles qu'aux utilisateurs connectés
ENF03	Sécurité	La session de l'utilisateur est maintenue de façon sécurisée
ENF04	Sécurité	L' espace administrateur est protégé séparément
ENF05	Performance	Les notifications sont vérifiées toutes les 3 secondes
ENF06	Qualité	Tous les formulaire s vérifient les données avant envoi
ENF07	Accessibilité	L'interface s'adapte aux écrans mobiles et ordinateurs
ENF08	Architecture	Les données partagées sont accessibles dans toute l'application
ENF09	Stockage	Les images des produits sont sauvegardées sur le serveur

Conclusion du chapitre

Ce chapitre a listé les fonctionnalités attendues de **Bloom** et les critères de qualité à respecter. Le chapitre suivant présente la conception de l'application.

CHAPITRE 3

Conception du Système

Introduction du chapitre

Ce chapitre explique comment **Bloom** est **organisée techniquement**. Il présente la structure générale de l'application, le fonctionnement de la connexion sécurisée, la protection des pages, la gestion des images et le système de notifications.

3.1 Architecture Générale de l'Application

Bloom suit une **architecture client-serveur** classique, divisée en **deux grandes parties** qui communiquent entre elles :

Vue d'ensemble de l'architecture

- **Le Frontend** (ce que l'utilisateur voit) : construit avec **React**, il s'exécute dans le navigateur. Il affiche les pages, gère les interactions de l'utilisateur et envoie des requêtes au serveur.
- **Le Backend** (le serveur) : construit avec **Node.js** et **Express**, il reçoit les requêtes du frontend, les traite, et renvoie les données demandées.
- **La Base de données** : **MongoDB** stocke toutes les données de l'application (utilisateurs, entrées de journal, produits, commandes, etc.).

Le schéma de communication est toujours le même :

1. L'utilisateur effectue une action dans le navigateur (clic, soumission de formulaire).
2. Le **frontend** envoie une requête **HTTP** au serveur via **Axios**.
3. Le **backend** reçoit la requête, vérifie les droits, et interroge **MongoDB**.
4. Le serveur renvoie une **réponse** (données ou confirmation) au frontend.

5. Le frontend **met à jour l’affichage** avec les nouvelles données.

Organisation du Frontend

Le frontend est structuré en **quatre couches distinctes** :

TABLE 3.1 – Organisation du frontend de Bloom

Dossier	Contenu	Rôle
<code>context/</code>	4 contextes React	Données globales partagées dans toute l’application
<code>components/</code>	Composants réutilisables	Éléments d’interface utilisés dans plusieurs pages
<code>layouts/</code>	3 mises en page	Structure visuelle de chaque espace (journal, boutique, admin)
<code>pages/</code>	20+ pages	Toutes les vues de l’application

Organisation du Backend

Le backend suit le **modèle MVC** (Modèle – Vue – Contrôleur), une organisation standard qui sépare clairement les responsabilités :

TABLE 3.2 – Organisation du backend de Bloom

Dossier	Contenu	Rôle
<code>models/</code>	Schémas Mon-goose	Définissent la structure des données stockées dans MongoDB
<code>controllers/</code>	Logique métier	Traitent les requêtes et répondent au frontend
<code>routes/</code>	Points d’entrée API	Définissent les adresses URL disponibles sur le serveur
<code>middlewares/</code>	Fonctions inter-médiaires	Vérifient les droits d’accès avant chaque requête

3.2 Organisation de l'Interface — Les Contextes React

Dans une application React, lorsque plusieurs pages ont besoin des mêmes données (comme l'identité de l'utilisateur connecté ou le nombre d'articles dans le panier), il faut trouver un moyen de **partager ces informations facilement**. La solution utilisée dans **Bloom** s'appelle les **contextes React** : ce sont des **espaces de données globaux**, accessibles depuis n'importe quelle page de l'application, sans avoir à les transmettre manuellement de page en page.

Bloom utilise **quatre contextes**, organisés du plus général au plus spécifique :

- **AuthContext** : gère l'**identité de l'utilisateur connecté**. C'est le contexte le plus important, car toute l'application dépend de savoir si l'utilisateur est connecté ou non.
- **CartContext** : suit le **nombre d'articles dans le panier**, affiché en permanence dans la barre de navigation.
- **NotificationContext** : interroge le serveur toutes les **3 secondes** pour vérifier si de nouvelles notifications sont arrivées, et met à jour le badge affiché.
- **ToastContext** : affiche des **messages de confirmation ou d'erreur** dans un bandeau qui apparaît brièvement à l'écran.

TABLE 3.3 – Les 4 contextes React de Bloom

Contexte	Rôle
AuthContext	Stocke les informations de l'utilisateur connecté et gère la connexion et la déconnexion .
CartContext	Maintient à jour le compteur du panier dans la barre de navigation.
NotificationContext	Vérifie toutes les 3 secondes si de nouvelles notifications sont disponibles.
ToastContext	Affiche des messages courts (succès, erreur, information) visibles quelques secondes.

3.3 Système de Connexion Sécurisée

Pour protéger l'accès à l'application, **Bloom** utilise un système d'authentification basé sur les **JSON Web Tokens (JWT)**. Le principe est simple : lorsqu'un utilisateur se connecte avec son adresse email et son mot de passe, le serveur vérifie ses informations et lui remet un **jeton d'identification**. Ce jeton est ensuite joint à chaque requête envoyée au serveur, comme une **carte d'accès numérique** qui prouve que l'utilisateur est bien qui il dit être.

Pour plus de sécurité, **Bloom** utilise **deux types de jetons** : un **jeton d'accès**, valable **15 minutes**, qui est utilisé pour naviguer dans l'application, et un **jeton de renouvellement**, valable **7 jours**, qui permet d'obtenir un nouveau jeton d'accès sans que l'utilisateur ait à se reconnecter. Les mots de passe, quant à eux, sont **chiffrés** avant d'être stockés grâce à la bibliothèque **bcrypt**, ce qui garantit qu'ils ne sont jamais lisibles directement.

TABLE 3.4 – Les deux types de jetons d'authentification

Jeton	Durée	Stockage	Usage
Jeton d'accès	15 minutes	Mémoire de l'application	Accès aux pages privées
Jeton de renouvellement	7 jours	Cookie du navigateur	Obtenir un nouveau jeton d'accès

Pourquoi chiffrer les mots de passe ?

Stocker un mot de passe **en clair** dans la base de données est très dangereux : si quelqu'un accède à la base, il peut lire tous les mots de passe directement. Le **chiffrement avec bcrypt** transforme le mot de passe en une suite de caractères illisible. Même si la base de données est compromise, les mots de passe réels restent **protégés**.

3.4 Protection des Pages

Bloom distingue deux types de pages : les **pages publiques** (connexion, inscription) et les **pages privées** (journal, boutique, profil, administration). Pour empêcher un utilisateur non connecté d'accéder aux pages privées, l'application utilise **deux mécanismes de protection** : un côté interface et un côté serveur.

Côté interface — ProtectedRoute

Le composant **ProtectedRoute** agit comme un **gardien** placé devant chaque page privée. Avant d'afficher une page, il vérifie si l'utilisateur est bien connecté. Si ce n'est pas le cas, il le **redirige automatiquement** vers la page de connexion. Si oui, il laisse la page s'afficher normalement.

Côté serveur — Vérification du jeton

Du côté serveur, chaque requête vers une route protégée passe par une **vérification du jeton d'accès**. Si le jeton est absent ou invalide, le serveur **refuse la requête** et renvoie une erreur. Ce **double niveau de protection** — côté interface et côté serveur — garantit qu'aucune donnée privée ne peut être consultée sans une connexion valide.

3.5 Upload d'Images Produits

Pour permettre aux administrateurs d'ajouter des photos aux produits de la boutique, **Bloom** intègre un **système d'upload de fichiers** grâce à la bibliothèque **Multer**. Lorsqu'un administrateur remplit le formulaire de création ou de modification d'un produit, il peut **sélectionner une image depuis son ordinateur**. L'image est alors envoyée au serveur, qui vérifie qu'il s'agit bien d'un fichier image (**JPEG, PNG ou WebP**) et la sauvegarde dans un **dossier dédié**. L'image est ensuite accessible et affichée dans la boutique.

Stockage local des images

Dans cette version de **Bloom**, les images sont sauvegardées **directement sur le serveur**. C'est une solution simple et efficace pour un projet de développement. Pour une application destinée à de nombreux utilisateurs, il serait préférable d'utiliser un service de stockage en ligne comme **Cloudinary** ou **Amazon S3**.

3.6 Système de Notifications

Bloom intègre un système de notifications qui permet d'**informer les utilisateurs et l'administrateur** des événements importants. Ce système fonctionne par **interrogation régulière** : toutes les **3 secondes**, l'application interroge automatiquement le serveur pour savoir s'il y a de nouvelles notifications, et met à jour un **badge** visible dans la barre de navigation.

Un mécanisme particulier concerne la **liste de souhaits** : lorsqu'un utilisateur ajoute un produit à sa wishlist, le serveur crée automatiquement une **alerte dans le panneau d'administration**. L'administrateur voit ainsi apparaître cette information en quasi-temps réel, sans avoir à recharger la page.

TABLE 3.5 – Composants gérant les notifications dans Bloom

Composant	Fréquence	Rôle
Contexte Notifications	toutes les 3s	Badge notifications journal
Layout Boutique	toutes les 3s	Badge alertes boutique
Layout Administration	toutes les 3s	Badge alertes admin
Page Notifications Admin	toutes les 3s	Liste complète des alertes

Conclusion du chapitre

Ce chapitre a présenté les principaux choix de conception de **Bloom** : l'**architecture client-serveur**, l'organisation des données partagées, le **système de connexion sécurisée**, la **protection des pages**, l'**upload d'images** et les **notifications automatiques**.

CHAPITRE 4

Réalisation et Mise en Œuvre

Introduction

Ce chapitre présente les principales fonctionnalités développées dans **Bloom** et les solutions apportées aux difficultés rencontrées.

4.1 Organisation du Projet

Le projet est structuré en **deux parties distinctes** : le **frontend** (interface utilisateur) et le **backend** (serveur et base de données).

Le frontend est organisé en plusieurs dossiers : les **contextes partagés**, les **composants réutilisables**, les **mis en page** (une par espace de l'application) et les **pages**. On y trouve plus de **vingt pages** réparties entre les espaces authentification, journal, boutique et administration.

Le backend est organisé selon une **séparation claire** entre les **modèles de données** (la structure des informations stockées dans MongoDB), les **contrôleurs** (la logique de traitement des requêtes) et les **routes** (les adresses URL des différentes fonctionnalités de l'API).

4.2 Validation des Formulaires

Tous les formulaires de **Bloom** sont soumis à une **validation avant d'être envoyés au serveur**. Cette validation est réalisée grâce à deux outils complémentaires : **react-hook-form**, qui gère l'état des champs du formulaire, et **Zod**, qui définit les **règles à respecter** (format d'email valide, mot de passe d'au moins **6 caractères**, champs obligatoires, etc.). Si une règle n'est pas respectée, un **message d'erreur s'affiche directement** sous le champ concerné, guidant l'utilisateur pour corriger sa saisie.

4.3 Expérience Utilisateur — Notifications Visuelles

Pour améliorer l'expérience utilisateur, **Bloom** affiche des **messages de confirmation ou d'erreur** après chaque action importante. Ces messages, appelés **toasts**, apparaissent brièvement en superposition de l'interface et disparaissent automatiquement. Ils permettent à l'utilisateur de savoir immédiatement si son action a réussi ou si une erreur s'est produite.

4.4 Gestion de l'Upload d'Image dans l'Administration

Lors de la création ou de la modification d'un produit, l'administrateur peut **joindre une image** en la sélectionnant depuis son ordinateur. Le formulaire envoie alors à la fois les **informations textuelles** du produit (nom, prix, description) et le **fichier image** dans la même requête. Le serveur reçoit le tout, **vérifie que le fichier est bien une image**, le sauvegarde, et enregistre le produit en base de données avec le chemin vers cette image.

4.5 Deux Espaces, Deux Systèmes de Connexion

TABLE 4.1 – Comparaison des deux systèmes d'authentification

	Utilisateurs	Administrateur
Accès	Jeton d'accès	Cookie de session
Stockage	Mémoire React	Navigateur (automatique)
Gestion	Manuelle	Automatique

Bloom distingue **deux systèmes de connexion indépendants**. Les **utilisateurs classiques** s'authentifient via un **jeton d'accès** transmis dans chaque requête. Les **administrateurs**, eux, utilisent un **cookie de session** géré automatiquement par le

navigateur. Cette séparation garantit que les deux espaces sont **totalelement isolés** l'un de l'autre, ce qui renforce la **sécurité globale** de l'application.

4.6 Difficultés Rencontrées et Solutions

Le développement de **Bloom** a été jalonné de plusieurs **problèmes techniques** qui ont nécessité analyse et recherche.

TABLE 4.2 – Principales difficultés et solutions apportées

Difficulté	Cause identifiée	Solution apportée
Page blanche après connexion	Redirection vers une adresse incorrecte	Correction du chemin de redirection
Images non affichées	Mauvais format d'URL lors du peuplement de la base	Utilisation d'URLs d'images au bon format
Panier vide après commande	Le panier n'était pas vidé après validation	Ajout d'une réinitialisation du panier à la confirmation
Notifications non reçues	Jeton d'accès expiré non renouvelé	Ajout du jeton dans chaque appel API concerné

Méthode de débogage adoptée

Face à chaque problème, la démarche suivie était toujours la même : **observer** ce qui se passe dans le navigateur, **lire attentivement** le message d'erreur affiché, **tester** le serveur avec l'outil Postman, puis **corriger** et vérifier. Cette méthode simple et systématique a permis de résoudre chaque bug efficacement.

4.7 Ce que ce Projet m'a Appris

Ce projet a été une expérience très enrichissante, tant sur le plan **technique** que **personnel**.

Compétences développées

Ce projet m'a permis de progresser sur plusieurs compétences techniques. J'ai appris à faire fonctionner ensemble une **interface**, un **serveur** et une **base de données**, ce qui constitue le cœur du développement full stack. J'ai également compris comment **protéger les données** des utilisateurs et sécuriser l'accès aux pages. La gestion des données partagées m'a amenée à utiliser les **contextes React** pour éviter de répéter les mêmes informations dans chaque composant. J'ai aussi appris à **valider les formulaires** avant envoi, à gérer l'**upload d'images** depuis un formulaire web, et à **déboguer** méthodiquement en lisant les messages d'erreur.

Leçons de méthode

Au-delà des compétences techniques, ce projet m'a enseigné des leçons de méthode précieuses. **Bien organiser son code dès le départ** évite de devoir tout réécrire plus tard. **Tester chaque fonctionnalité séparément** avant de tout assembler empêche les erreurs de se multiplier. **Lire les messages d'erreur attentivement** est souvent suffisant pour trouver la solution. Enfin, **persévérer** face aux difficultés est une compétence à part entière, aussi importante que les compétences techniques elles-mêmes.

Conclusion du chapitre

Ce chapitre a présenté les principales fonctionnalités développées dans **Bloom**, les difficultés rencontrées et les solutions apportées.

Conclusion Générale

Bloom est une **application web complète** développée dans le cadre du cours de Développement Full Stack à l'EPI Digital School de Sousse. Elle réunit en un seul outil **trois espaces fonctionnels distincts** : un journal personnel intime, une boutique en ligne et un panneau d'administration.

Ce projet met en pratique les **concepts fondamentaux du cours** : connexion sécurisée, validation des formulaires, gestion de l'état global, protection des pages, communication avec un serveur et stockage en base de données. Il va au-delà du minimum exigé en intégrant des fonctionnalités supplémentaires comme la **protection des entrées par code PIN**, l'**upload d'images**, les **notifications en temps quasi-réel** et une **liste de souhaits** connectée au panneau d'administration.

Bilan du projet

Bloom est une application de plus de **20 pages** réparties entre le journal, la boutique et l'administration. Elle intègre une **connexion sécurisée** à deux niveaux de jetons, des **formulaires tous validés** avant envoi au serveur, et un **panneau d'administration** indépendant et complet. Les produits disposent d'un système d'**upload d'images**, les utilisateurs reçoivent des **notifications automatiques** toutes les 3 secondes, et chaque ajout à la wishlist déclenche une **alerte admin** en temps quasi-réel. Enfin, le journal intime permet de **protéger les entrées sensibles** par un code PIN personnel.

Améliorations Envisageables

Bloom constitue une **base solide** qui pourrait être améliorée dans plusieurs directions :

- **Notifications instantanées** : remplacer l'interrogation régulière par un système de connexion permanente entre le navigateur et le serveur, pour des alertes encore plus réactives.

- **Stockage des images en ligne** : utiliser un service de stockage cloud pour les photos des produits, ce qui rendrait l'application plus facile à déployer.
- **Paiement en ligne** : intégrer un système de paiement sécurisé pour compléter le tunnel d'achat.
- **Application mobile** : adapter **Bloom** pour smartphones et tablettes grâce à React Native.
- **Tests automatisés** : ajouter des tests pour vérifier automatiquement que les fonctionnalités restent opérationnelles après chaque modification.

Ce projet m'a permis de progresser concrètement dans le **développement web** et de comprendre comment construire une **application complète de bout en bout**. **Bloom** est pour moi non seulement un travail académique, mais aussi une **vraie réalisation** dont je suis fière.

Bloom — Application Web Full Stack

React • Node.js • MongoDB • JWT • Multer • Zod

EPI Sousse — 2025/2026 — Ons GAAYA